

Contents

guide Proximal Policy Optimization Algorithms	1
0. 这篇论文到底解决什么	1
1. 回到 2017 年的语境	2
2. 原论文阅读地图	2
3. 从 policy gradient 开始	2
4. TRPO 给出的答案和代价	3
5. PPO-Clip 的核心公式	3
6. $A > 0$ 时 clip 在干什么	3
7. $A < 0$ 时 clip 在干什么	4
8. 最小代码实现	4
9. 为什么能多轮 minibatch 更新	5
10. Value loss、entropy 和 GAE	6
11. Adaptive KL penalty 是什么	6
12. 本地 CartPole PPO 怎么对上	7
13. 本地 GPT-2 PPO 怎么对上	7
14. 实验证据链	8
15. PPO 的理论直觉	8
16. PPO 和 RLHF 的关系	9
17. 为什么 DPO 会出现	9
18. 30 分钟本地实验	9
19. 常见误区	10
20. 闭卷掌握检查	10
21. 用 AI agent 学这篇的正确方式	11

guide_Proximal Policy Optimization Algorithms

原论文: Proximal Policy Optimization Algorithms

本地原文 PDF: [learning/rl-foundations/paper/01_proximal_policy_optimization.pdf](#)

作者: Schulman et al.

年份: 2017

类型: paper

0. 这篇论文到底解决什么

PPO 解决的是 policy gradient 里最常见的痛点: 更新太小, 学得慢; 更新太大, policy 崩。TRPO 可以用 trust region 控制 policy update, 但实现复杂, 需要二阶近似、共轭梯度和约束优化。PPO 的目标是保留 TRPO 的稳定性, 同时只用一阶优化和普通 minibatch SGD/Adam。

一句话:

PPO =
on-policy rollout
+ advantage estimate
+ clipped probability ratio
+ multiple minibatch epochs

这篇论文对后来的 LLM-RLHF 极其重要。InstructGPT 里的 PPO，不是神秘的“让模型变好”的魔法，而是这篇论文的思想移植到 token-level policy 上：

```
old policy samples response
new policy tries to improve reward
ratio = pi_new(token) / pi_old(token)
clip ratio to avoid too large update
add value loss, entropy, KL/reference constraints
```

理解 PPO 后，再看 RLHF 里的 actor、critic、reference、reward model、KL penalty、advantage normalization，就不再是一团术语。

1. 回到 2017 年的语境

当时深度强化学习已有几条主线：

- DQN/Q-learning: 对离散动作很强，但在函数逼近、连续控制和稳定性上有局限。
- Vanilla policy gradient / A2C: 简单、可扩展，但样本效率和鲁棒性不足。
- TRPO: 更稳定，使用 trust region 约束更新幅度，但实现复杂，也不太适合 dropout、共享参数或辅助任务等结构。

PPO 论文的 design taste 很工程化：不追求最漂亮的理论约束，而是找一个足够稳、足够简单、能在很多任务上跑起来的近似。它提出的 clipped surrogate objective，是把“trust region”变成一个可以直接写成 loss 的局部规则。

这也是为什么 PPO 后来成为 RLHF 默认基线之一。它不是最优雅的计算，但它足够通用，能和神经网络、并行 rollout、value head、minibatch、Adam、entropy bonus 等工程组件拼起来。

2. 原论文阅读地图

建议按这个顺序读：

1. Abstract/Introduction: 看 PPO 想同时要 sample efficiency、simplicity 和 wall-time。
2. Section 2: 看 policy gradient objective 和为什么同一 trajectory 上多步更新会危险。
3. Section 3: 看 clipped surrogate objective，这是核心。
4. Figure 1: 分别看 positive advantage 和 negative advantage 时 ratio clipping 的形状。
5. Figure 2: 看 clipped objective 如何成为 unconstrained surrogate 的 pessimistic lower bound。
6. Section 4: 看 adaptive KL penalty，这是替代方案和重要对照。
7. Section 5: 看 value loss、entropy bonus、GAE、N actors x T timesteps x K epochs。
8. Section 6: 看 MuJoCo、Roboschool、Atari 的实验。
9. Appendix hyperparameters: 看 horizon、epochs、minibatch、gamma、lambda、clip epsilon 等实际取值。

如果只读一次，必须读懂 Equation 7。读懂它，PPO 这个名字就有了身体。

3. 从 policy gradient 开始

Vanilla policy gradient 的核心估计是：

$$g = E_t [\text{grad}_{\theta} \log \pi_{\theta}(a_t | s_t) * A_t]$$

其中：

- $\pi_{\theta}(a_t | s_t)$: 当前 policy 在状态 s_t 下选动作 a_t 的概率。
- A_t : advantage estimate，表示这个动作比平均水平好多少。

- 如果 $A_t > 0$, 希望增加这个动作概率。
- 如果 $A_t < 0$, 希望降低这个动作概率。

自动微分实现时, 可以构造一个 surrogate objective:

```
L_PG(theta) = E_t [ log pi_theta(a_t | s_t) * A_t ]
```

问题是: on-policy 数据来自旧 policy。如果用同一批 trajectory 对 L_{PG} 做很多步更新, 新 policy 很快就离采样 policy 很远, 数据不再可信, 更新可能变成灾难。

PPO 的核心问题就是:

怎样重复利用一批 on-policy 数据, 又不让 policy 一步走太远?

4. TRPO 给出的答案和代价

TRPO 的思想是最大化 surrogate objective, 同时约束新旧 policy 的 KL divergence:

```
maximize    E_t [ (pi_theta(a_t|s_t) / pi_old(a_t|s_t)) * A_t ]
subject to  E_t [ KL(pi_old(.|s_t), pi_theta(.|s_t)) ] <= delta
```

这里的 ratio:

```
r_t(theta) = pi_theta(a_t | s_t) / pi_old(a_t | s_t)
```

如果 $r_t = 1$, 说明新旧 policy 对这个动作的概率一样。如果 $r_t = 1.3$, 说明新 policy 把这个动作概率提高了 30%。如果 $r_t = 0.7$, 说明降低了 30%。

TRPO 的约束很合理, 但实现代价高。PPO 问的是: 能不能不用显式约束优化, 只通过一个普通 loss 近似这个 “不走太远” 的要求?

5. PPO-Clip 的核心公式

PPO clipped objective:

```
L_CLIP(theta) =
  E_t [
    min(
      r_t(theta) * A_t,
      clip(r_t(theta), 1 - eps, 1 + eps) * A_t
    )
  ]
```

常见 $\text{eps} = 0.2$ 。这意味着 ratio 的 “有奖励更新区间” 大致是:

```
0.8 <= r_t <= 1.2
```

关键是 min。它让 objective 成为更保守的版本, 而不是简单把 ratio 截断。论文称它是 pessimistic bound 的直觉: 当 update 继续变大会让目标看起来更好时, clip 不再给它额外奖励; 当 update 变坏时, 仍然让它承担损失。

6. $A > 0$ 时 clip 在干什么

假设 $A_t > 0$, 动作比平均好, policy 应该提高这个动作概率。

```

unclipped = r * A
clipped   = clip(r, 1-eps, 1+eps) * A
objective = min(unclipped, clipped)

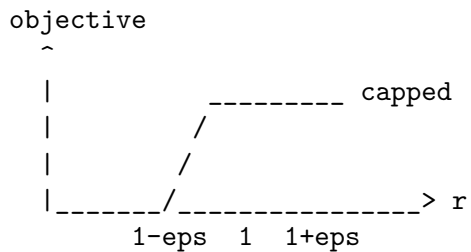
```

因为 $A > 0$:

- 当 r 从 1 增加到 $1+\epsilon$, objective 增加, 鼓励提高好动作概率。
- 当 $r > 1+\epsilon$, clipped 项固定在 $(1+\epsilon)A$, 继续提高概率不会再得到额外收益。
- 当 $r < 1-\epsilon$, unclipped 更小, objective 会惩罚把好动作概率降太多。

图像直觉:

$A > 0$



所以它不是禁止更新, 而是不给“过度提高好动作概率”额外甜头。

7. $A < 0$ 时 clip 在干什么

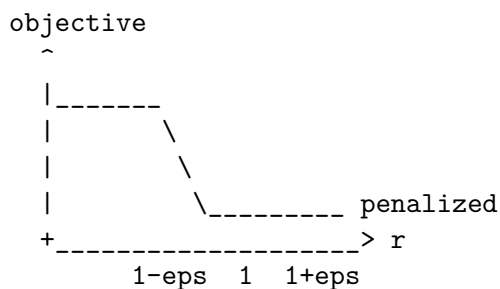
假设 $A_t < 0$, 动作比平均差, policy 应该降低这个动作概率。

因为 $A < 0$, 乘法方向反过来:

- 当 r 从 1 降到 $1-\epsilon$, objective 变好, 鼓励降低坏动作概率。
- 当 $r < 1-\epsilon$, clipped 项固定, 继续降低概率不会再得到额外收益。
- 当 $r > 1+\epsilon$, unclipped 变得更差, 惩罚把坏动作概率提高太多。

图像直觉:

$A < 0$



这就是 Figure 1 要你看到的东西: positive advantage 和 negative advantage 的 clipping 方向不同。很多初学者背了公式, 但没想清楚这两个分支, 结果实现 PPO 时不知道 loss 是否合理。

8. 最小代码实现

PPO-Clip 的核心代码只有几行:

```
ratio = torch.exp(logp_new - logp_old)
surr1 = ratio * advantage
surr2 = ratio.clamp(1 - eps, 1 + eps) * advantage
policy_loss = -torch.min(surr1, surr2).mean()
```

负号是因为 PyTorch optimizer 默认最小化 loss，而论文写的是最大化 objective。

张量级别:

```
obs:          [B, obs_dim]
actions:      [B]
logp_old:     [B]
logp_new:     [B]
ratio:        [B]
advantage:    [B]
surr1:        [B]
surr2:        [B]
loss:         scalar
```

在 LLM token-level PPO 中，动作是 token:

```
input_ids:    [B, T]
logp_old:     [B, T-1]
logp_new:     [B, T-1]
advantage:    [B, T-1]
mask:         [B, T-1]
loss = masked mean over response tokens
```

PPO 从 CartPole 到 LLM 的形状变化很大，但 ratio 和 clip 的逻辑没有变。

9. 为什么能多轮 minibatch 更新

Vanilla policy gradient 通常是一批数据做一次更新。PPO 允许:

```
collect rollout with pi_old
compute logp_old and advantages
for epoch in 1..K:
    shuffle rollout batch
    for minibatch:
        compute logp_new
        optimize clipped objective
```

关键是 logp_old 固定。它记录采样时的 policy 概率。每次更新时，新 policy 都和旧 policy 比 ratio。

论文 Algorithm 1 的结构:

```
for each iteration:
    for each of N parallel actors:
        run pi_old for T timesteps
    compute advantage estimates on N*T samples
    optimize surrogate objective for K epochs with minibatch size M
    pi_old <- pi_new
```

这也是 PPO 的样本效率来源: 它不是每条 trajectory 只用一次，而是在 clipping 保护下用多次。

10. Value loss、entropy 和 GAE

实际 PPO 不只优化 policy loss。actor-critic 版本通常最大化:

```
L_total =  
    L_clip  
    - c1 * value_loss  
    + c2 * entropy_bonus
```

如果写成 PyTorch 最小化:

```
loss =  
    policy_loss  
    + c1 * value_loss  
    - c2 * entropy
```

Value loss:

```
value_loss = (V_theta(s_t) - return_t)^2
```

Entropy bonus:

```
entropy = - sum_a pi(a|s) log pi(a|s)
```

它鼓励探索, 防止 policy 太快塌成确定性。

GAE 用 TD residual 累积 advantage:

```
delta_t = r_t + gamma * V(s_{t+1}) - V(s_t)  
A_t = delta_t + gamma*lambda*delta_{t+1}  
      + (gamma*lambda)^2*delta_{t+2} + ...
```

本仓库 compute_gae 的代码就是反向递推:

```
for t in reversed(range(T)):  
    mask = 1.0 - dones[t].float()  
    delta = rewards[t] + gamma * next_value * mask - values[t]  
    gae = delta + gamma * lam * mask * gae  
    advantages[t] = gae  
    next_value = values[t]  
returns = advantages + values
```

你可以把 GAE 看成 bias-variance knob:

- $\lambda=1$: 更接近 Monte Carlo return, 方差更大。
- $\lambda=0$: 更接近 one-step TD, 偏差更大。
- 常用 $\lambda=0.95$ 。

11. Adaptive KL penalty 是什么

论文还讨论了一个替代方案: 不用 clip, 而是在 objective 里加 KL penalty:

```
L_KL(theta) =  
    E_t [  
        r_t(theta) * A_t  
        - beta * KL(pi_old(.|s_t), pi_theta(.|s_t))  
    ]
```

然后根据实际 KL 调整 beta:

```
if KL < target / 1.5:
    beta = beta / 2

if KL > target * 1.5:
    beta = beta * 2
```

论文实验中 adaptive KL penalty 不如 clipped objective, 但它很重要, 因为 RLHF 里的 reference KL 控制和这里有血缘关系。InstructGPT 的 $\text{reward} - \beta \text{KL}(\pi_{\text{RL}} \parallel \pi_{\text{SFT}})$ 更像 “优化 reward, 同时不要跑离 reference 太远”。

PPO paper 的核心结论不是 “KL 没用”。更准确地说: 对普通 PPO benchmark, clip objective 的工程表现最好; KL penalty 是重要对照和补充。

12. 本地 CartPole PPO 怎么对上

本仓库 `learning/rl-foundations/src/ppo_minimal.py` 对应论文 actor-critic PPO:

- PolicyValue: 共享 backbone, 分出 actor logits 和 critic value。
- Rollout buffer: 保存 obs/actions/logp_old/values/rewards/dones。
- compute_gae: 根据 rewards 和 values 得到 advantages/returns。
- Advantage normalization: PPO 常用 trick, 让更新尺度更稳定。
- $\text{ratio} = \exp(\log p_{\text{new}} - \log p_{\text{old}})$: 论文的 probability ratio。
- $\text{ratio.clamp}(1-\epsilon, 1+\epsilon)$: clipped surrogate。
- L_{vf} : value function squared error。
- entropy: exploration bonus。
- K_{epochs} 和 minibatches: 同一 rollout 重复优化。

核心代码对应:

```
ratio = (logp_new - logp_old_mb).exp()
surr1 = ratio * adv_mb
surr2 = ratio.clamp(1 - args.eps, 1 + args.eps) * adv_mb
L_clip = -torch.min(surr1, surr2).mean()

L_vf = F.mse_loss(V_new, ret_mb)
loss = L_clip + args.vf_coef * L_vf - args.ent_coef * entropy
```

如果 PPO 跑不起来, 先看这些监控:

- episode return 是否上升。
- approximate KL 是否爆掉。
- entropy 是否过快下降。
- value loss 是否持续很大。
- clip fraction 是否极端。
- advantage 是否 normalized。
- old logprob 是否在 rollout 后固定。

13. 本地 GPT-2 PPO 怎么对上

`learning/rl-foundations/src/ppo_gpt2_minimal.py` 把 PPO 映射到 token:

```
state: prefix tokens
action: next token
policy: GPT-2 actor
critic: GPT-2 + value head
old policy: rollout 时的 actor snapshot
reference policy: frozen GPT-2
reward: toy length reward or later RM score
mask: response tokens only
```

它的 token reward:

```
reward_t = - beta * (logp_actor_t - logp_ref_t)
final response token gets + raw_reward
```

然后对 response token 做 GAE 和 PPO clip:

```
ratio_t = exp(logp_new_t - logp_old_t)
loss_t = - min(ratio_t A_t, clip(ratio_t) A_t)
```

这就是从 PPO paper 到 InstructGPT 的桥。差异在于:

- CartPole 一步动作是离散 action, LLM 一步动作是 token。
- CartPole reward 来自环境, LLM reward 通常来自 RM。
- LLM 还需要 reference KL, 防止语言分布跑偏。
- LLM 需要 response mask, 否则 prompt token 也会被错误优化。

14. 实验证据链

论文用三类实验支持 PPO:

1. Surrogate objective ablation:
 - 比较 no clipping、fixed KL、adaptive KL、clipped objective 等版本。
 - 结论: clipped probability ratio 的版本表现最好。
2. Continuous control:
 - 在 MuJoCo 的多个模拟机器人任务上比较 PPO、TRPO、A2C、CEM、vanilla policy gradient 等。
 - PPO 在几乎所有 continuous control environments 上优于对照方法。
3. Roboschool 和 Atari:
 - Roboschool humanoid tasks 展示 PPO 能训练复杂 3D 控制。
 - Atari 49 games 中, PPO 在 sample complexity 上明显优于 A2C, 和 ACER 接近但更简单。

实验结论的重点不是“PPO 永远最好”, 而是它在 simplicity、sample complexity、robustness、wall-time 之间取得了很好的工程平衡。

15. PPO 的理论直觉

PPO 不是严格保证单调提升的算法。它的 clipped objective 是一种局部保守近似:

```
if update helps too much by moving ratio far from 1:
    stop giving extra objective reward
```

```
if update hurts:
    let the loss see the hurt
```

所以 PPO 的安全边界来自多层工程机制:

- Ratio clip 限制单样本 probability ratio 的收益。
- Advantage normalization 控制梯度尺度。
- Value loss 提供 baseline 和 critic 学习。
- Entropy bonus 保持探索。
- Optional KL monitoring 防止整体 policy drift。
- Minibatch + K epochs 提高样本利用。
- Gradient clipping 提升数值稳定性。

这也是为什么 PPO implementation details 很重要。只写出 clipped formula 不等于写出了稳定 PPO。

16. PPO 和 RLHF 的关系

InstructGPT 的 PPO 阶段可以这样看:

PPO paper:

environment reward + clip ratio + value function

InstructGPT:

reward model score
+ per-token KL penalty to SFT reference
+ PPO clip ratio
+ value function

在 LLM 里, policy update 风险更高:

- action space 是 vocabulary, 巨大。
- episode 是整段 response, credit assignment 难。
- reward model 是代理目标, 会被 hack。
- 语言质量会因为 KL 失控而快速退化。
- rollout 成本很高, batch 形状复杂。

所以 RLHF 中的 PPO 往往比 CartPole PPO 多很多监控: KL to reference、reward model score、length、entropy、clip fraction、value loss、human spot check。

17. 为什么 DPO 会出现

PPO-RLHF 难在:

- 要训练 reward model。
- 要 rollout。
- 要维护 actor、critic、reference、RM 多模型。
- 要调 KL。
- 要防 reward hacking。
- 工程成本高。

DPO 等 preference optimization 方法就是在问: 如果 PPO 的目标本质是 KL-regularized reward maximization, 能不能直接从 preference pairs 训练 policy, 绕开显式 RM 和 online PPO?

因此, PPO 仍然是理解 DPO 的父问题。DPO 简化了训练流程, 但它没有消灭 preference data 的偏差, 也没有消灭 reference policy 和 beta 这类控制问题。

18. 30 分钟本地实验

实验 1: 看 PPO clip 的分支。

```
import torch

eps = 0.2
r = torch.tensor([0.5, 0.8, 1.0, 1.2, 1.8])
A_pos = torch.ones_like(r)
A_neg = -torch.ones_like(r)

def ppo_term(r, A):
    return torch.min(r * A, r.clamp(1-eps, 1+eps) * A)

print("A>0", ppo_term(r, A_pos))
print("A<0", ppo_term(r, A_neg))
```

你应该看到: $A>0$ 时高 ratio 被 cap; $A<0$ 时低 ratio 被 cap。

实验 2: 跑 CartPole PPO。

```
python learning/rl-foundations/src/ppo_minimal.py --total-steps 100000
```

看三件事:

- episode mean 是否上升到接近 500。
- KL 是否稳定在小范围。
- entropy 是否逐步下降而不是瞬间归零。

实验 3: 改 eps。

eps = 0.05: 更新保守, 可能慢。

eps = 0.2: 常见默认。

eps = 0.5: 更新激进, 可能不稳定。

19. 常见误区

1. 误区: PPO clip 限制了 policy 的实际 KL。
 - 更准确: clip 限制的是 sampled action probability ratio 的 objective incentive, 不等于严格 KL 约束。
2. 误区: PPO 是 off-policy。
 - 更准确: PPO 是 on-policy, 但通过多 epoch minibatch 重复利用最近一批 on-policy 数据。
3. 误区: `logp_old` 可以训练时重新算。
 - 更准确: `logp_old` 必须是 rollout 时旧 policy 对已采样动作的 logprob。
4. 误区: Advantage 不需要 normalization。
 - 更准确: 不是理论必需, 但实践中非常常用, 能显著稳定梯度尺度。
5. 误区: LLM-RLHF 里的 PPO 和 CartPole PPO 完全不同。
 - 更准确: 环境和张量形状不同, 但 ratio、clip、advantage、value loss 的骨架相同。

20. 闭卷掌握检查

1. 为什么同一批 trajectory 上对 vanilla policy gradient 做很多步更新会危险?
2. TRPO 的 trust region 约束是什么? PPO 为什么想绕开它?
3. `r_t(theta)` 的定义是什么?
4. 写出 PPO clipped objective。
5. 当 $A_t > 0$ 时, $r > 1+eps$ 为什么不再给额外收益?
6. 当 $A_t < 0$ 时, $r < 1-eps$ 为什么不再给额外收益?

7. $\log p_{\text{old}}$ 和 $\log p_{\text{new}}$ 分别什么时候计算?
8. 为什么 PPO 可以对同一 rollout 做 K epochs minibatch 更新?
9. Value loss 和 entropy bonus 各自解决什么问题?
10. GAE 中 λ 控制什么 trade-off?
11. Adaptive KL penalty 和 clipped objective 有什么关系?
12. LLM-RLHF 比 CartPole PPO 多了哪些复杂性?

21. 用 AI agent 学这篇的正确方式

不要让 agent 只给你 “PPO 是一种 policy optimization 算法” 的定义。更好的 prompt 是:

我正在读 PPO 原论文。请你先用一个离散动作例子解释 $\text{ratio} = \pi_{\text{new}}/\pi_{\text{old}}$ 。

然后给我 $A > 0$ 和 $A < 0$ 两个分支, 让我手算 $\epsilon = 0.2$ 时 $r = 0.5, 0.8, 1.0, 1.2, 1.8$ 的 clipped objective。

接着让我写出 PyTorch 版 $\text{ratio}/\text{surr1}/\text{surr2}/\text{loss}$ 。

最后把 CartPole PPO 的 [B] shape 映射到 LLM token-level PPO 的 [B,T] shape。

如果我说 PPO 是严格 trust region, 请纠正并解释它只是 clipped surrogate approximation。

真正掌握 PPO 的标志是: 你能不用背公式, 直接从 “旧 policy 采样的数据不能被新 policy 过度消费” 推出 ratio、clip、advantage 和 K epochs; 能解释正负 advantage 的 clipping 方向; 能把 CartPole 的动作级 PPO 映射到 LLM 的 token-level PPO。